```
title     Platform Architecture
subtitle  C4 reference for the platform-template monorepo
date      2026-05-16
```

Platform Architecture

A C4-style reference for the `platform-template` monorepo. Describes the **target state** after the boilerplate-collapse, header-contract, and bouncer-attestation cleanups. The code may not match this document in some places yet; this document is the spec the cleanup converges to.

The platform is a Cloudflare Workers monorepo. Public traffic enters through a single Worker called **bouncer**. Bouncer resolves the user's session, stamps a signed attestation onto the forwarded request, and dispatches to one of several upstream Workers (apps or internal services) via service bindings. Apps trust bouncer's attestation, skip the redundant session lookup, and render. The session authority — Better Auth + the user database — lives in a Worker called **guestlist**, which only ever receives traffic over service bindings.

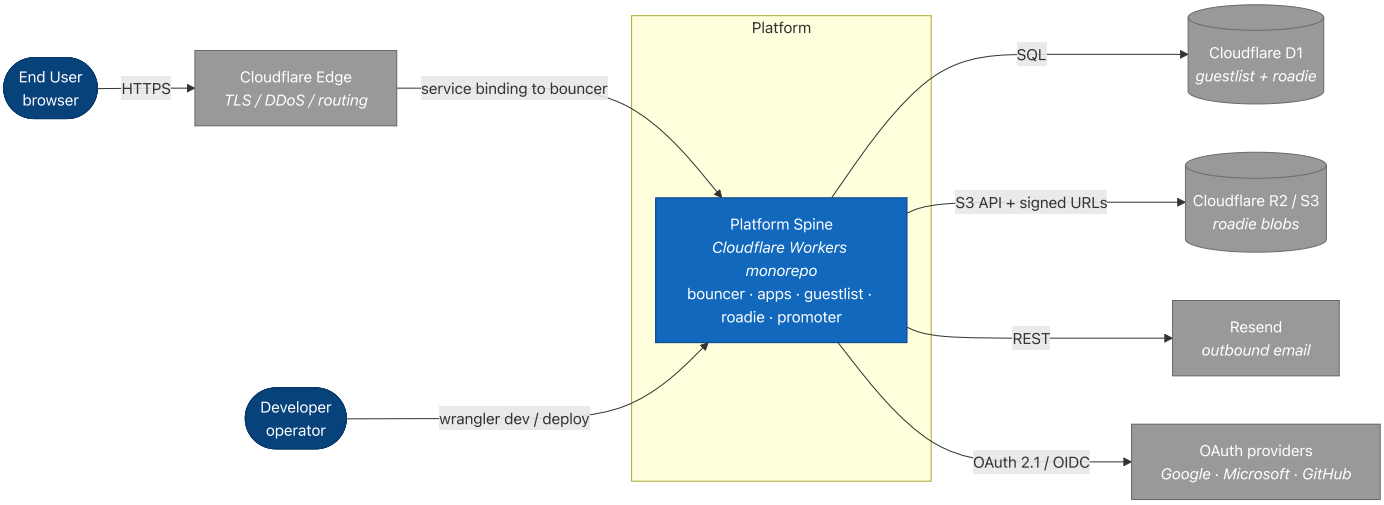
The template is opinionated about **TanStack Start** apps but the underlying primitives are framework-agnostic. You can drop in a Hono Worker, a plain `fetch` handler, or an Astro-on-Workers app and consume the same shared primitives (envelope verifier, guestlist client, request context, canonical log). The Start-specific helpers (`createPlatformStartApp` , `createPlatformStartWorkerEntry`) layer on top.

Dev/prod parity is explicit. Apps run alone in development (no bouncer in front, just service bindings to guestlist) and the same code paths route to guestlist. The same apps in production require a valid bouncer attestation envelope or return 403. The behavioral split is two `ENVIRONMENT` checks, both encoded inside `@platform/auth` 's verifier — never scattered across app code.

---

§1 — Context

The system from outside in.



Actors.

- **End User** — a browser. Uses one or more apps under the platform's `baseDomain` (e.g., `identity.platform.example`, `thoughts.platform.example`). One Better Auth cookie scoped to the apex domain authenticates all apps.
- **Developer / Operator** — the person running the monorepo. Runs `bun run dev` locally or `wrangler deploy` to ship. Operates the platform; not an end user of the apps it hosts.

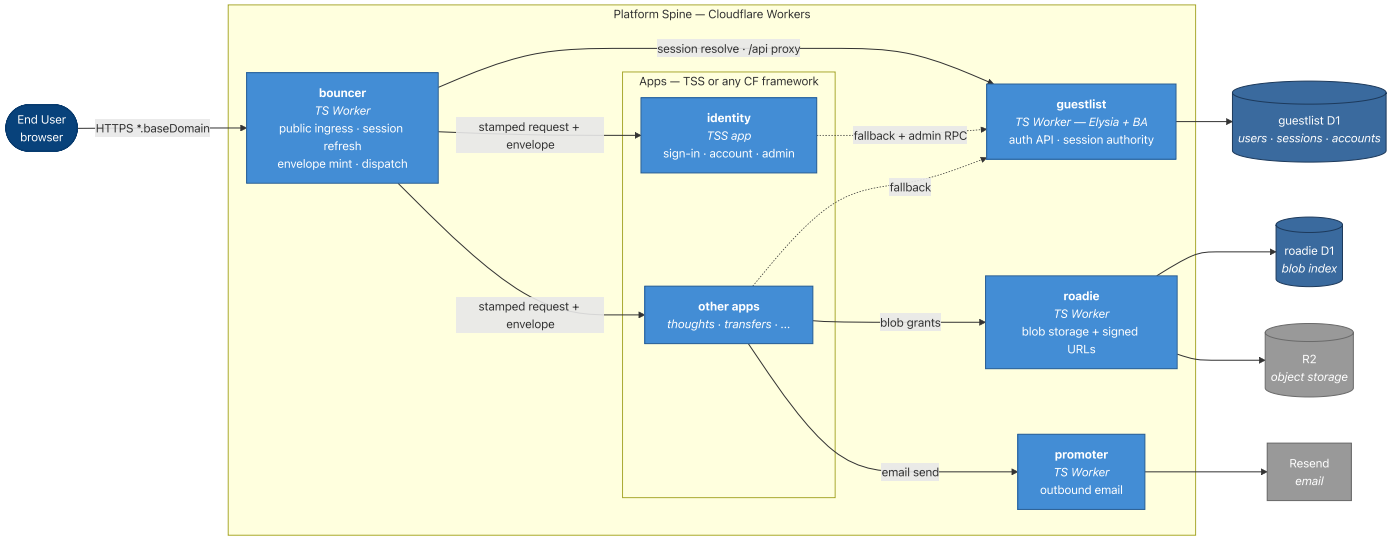
External dependencies.

- **Cloudflare Edge** — TLS termination + DDoS + request routing. Every public host in the platform is a CF Custom Domain pointing at the bouncer Worker.
- **Cloudflare D1** — relational store. Guestlist owns the auth schema (users, sessions, accounts, two-factor, organizations once enabled). Roadie owns the blob index. No app directly accesses any D1.
- **Cloudflare R2 / S3-compatible storage** — roadie's blob backend. App workers never touch R2 directly; they request signed PUT/GET URLs from roadie.
- **Resend** — promoter's email backend. App workers never call Resend directly.
- **OAuth providers** — Google, Microsoft, GitHub, LinkedIn, Facebook. Optional. Wired in guestlist's `auth-config.ts` when client ids/secrets are present in env.

---

§2 — Containers

The deployed Workers and what they do.

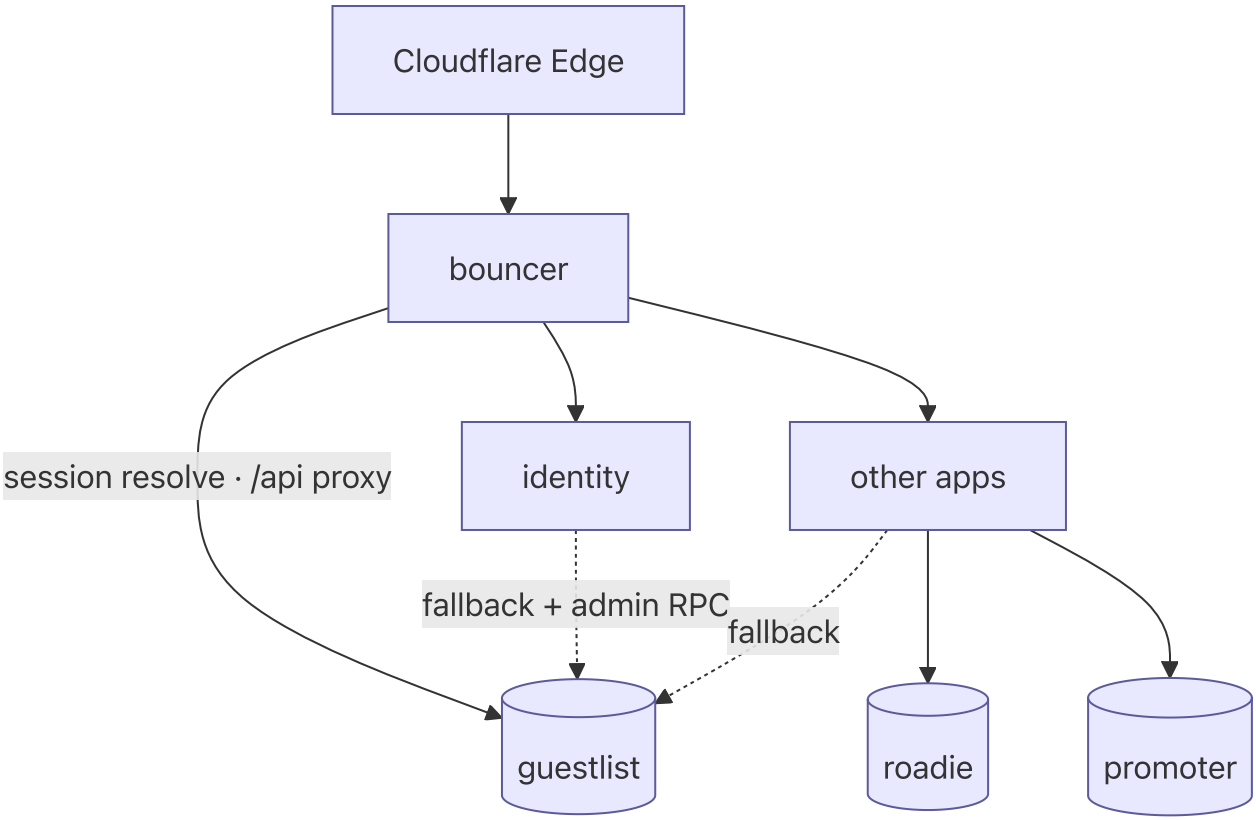

Platform Spine — Cloudflare Workers
End User browser
HTTPS *.baseDomain
bouncer
TS Worker
public ingress · session refresh
envelope mint · dispatch
session resolve · /api proxy
stamped request + envelope
Apps — TSS or any CF framework
identity
TSS app
sign-in · account · admin
fallback + admin RPC
guestlist
TS Worker — Elysia + BA
auth API · session authority
guestlist D1
users · sessions · accounts
fallback
stamped request + envelope
other apps
thoughts · transfers · ...
blob grants
roadie
TS Worker
blob storage + signed URLs
roadie D1
blob index
R2
object storage
email send
promoter
TS Worker
outbound email
Resend
email

§2.1 Containers in detail

Container	Role	Public host?	Holds secrets?
bouncer	Single public ingress. Resolves session via guestlist. Mints + stamps bouncer attestation envelope. Dispatches to apps (passthrough or mounted-microfrontend mode). Strips privileged headers from inbound and outbound traffic.	Yes — every public hostname is a bouncer Custom Domain.	`BNC_ATT_PRIV` (Ed25519 private key for envelope signing). No `BETTER_AUTH_SECRET`.
guestlist	Sole authority on session validity. Owns Better Auth wiring, the user database, plugin set (passkey, twoFactor, oauthProvider, admin). Exposes `/api/auth/*` (BA handler), `/api/avatar/*` (presigned-upload flow via roadie), `/admin/*` (sessions, stats, API keys, OAuth clients), `/user/connections`, `/u/avatar/:refId` (public avatar read broker), `/providers`, `/.well-known/*`, `/health`.	No public Custom Domain in the target topology. Reached only via service bindings (from bouncer for `/api` / `/u`, from apps for session/admin).	`BETTER_AUTH_SECRET`, OAuth client secrets, internal API tokens.
roadie	Blob storage and signed-URL minting for app-uploaded files. Apps request "give me a PUT URL for blob X for user Y"; roadie validates and returns a presigned R2 URL.	No public Custom Domain.	`S3_ACCESS_KEY_ID`, `S3_SECRET_ACCESS_KEY`, signed-meta secret.
promoter	Outbound transactional email. Wraps Resend behind a typed RPC surface (`sendVerificationEmail`, `sendMagicLinkEmail`, etc.). Apps don't speak to Resend; they speak to promoter over a service binding.	No public Custom Domain.	`RESEND_API_KEY`, signed-meta secret.
identity (app)	Sign-in, sign-up, account settings, admin sessions surface. The reference TanStack Start app.	No direct host. Routes `identity.<baseDomain>` are bouncer → identity.	None at the app level. `BOUNCER_ATTESTATION_KEYS` is committed code (public keys), not a secret.
other apps	Product-specific. Each app reaches the same primitives via the same kit factories. May be TSS or any other CF-deployable framework.	Each gets a host under `<app>.<baseDomain>`, owned by bouncer.	None.

§2.2 Service binding graph

The trust graph is a DAG:



- **Bouncer** binds to every app and to guestlist. Bouncer never binds to roadie or promoter (no use case).
- **Apps** bind to guestlist (session fallback + `/api/auth` proxy), to roadie (blob grants), and to promoter (email).
- **Apps** do not bind to each other. App-to-app communication, when needed, is mediated by bouncer or by a shared service.
- **Guestlist, roadie, promoter** do not bind to anything inside the platform — they're leaves.

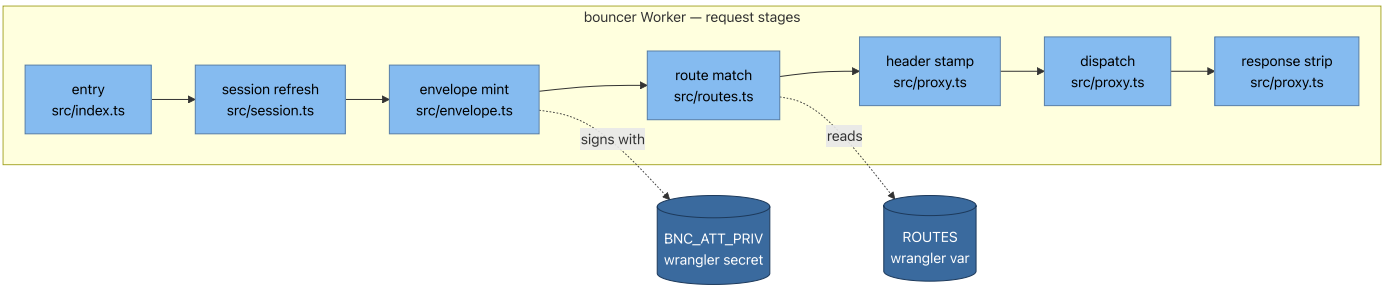
This shape is what makes the security model tractable: the platform's only inbound surface from the public Internet is bouncer, and the platform's authority chain converges on guestlist.

---

§3 — Components

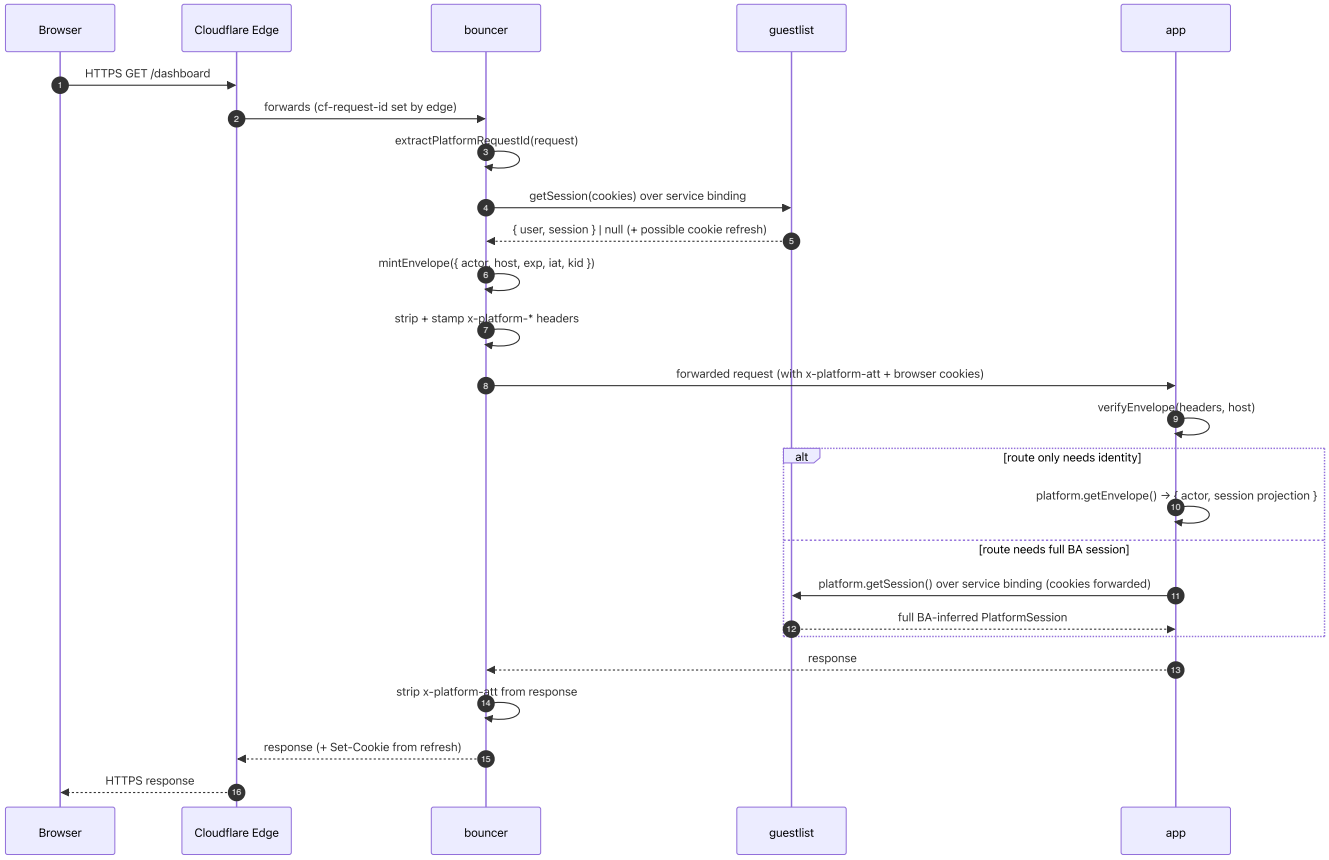
This section drops one level deeper into each Worker. Component diagrams + the load-bearing files.

§3.1 bouncer



stage	file	what it does
`entry`	`src/index.ts`	fetch handler; opens request context; runs stages in order
`session refresh`	`src/session.ts`	calls `guestlist.getSession()` over service binding; captures actor, session projection, refresh cookies
`envelope mint`	`src/envelope.ts`	builds Ed25519-signed attestation payload from resolved `{ actor, session }`
`route match`	`src/routes.ts`	compiled host+path matcher; produces a route binding + mode
`header stamp`	`src/proxy.ts`	strips inbound `x-platform-*`; stamps bouncer-authored values + the envelope
`dispatch`	`src/proxy.ts`	forwards to upstream via service binding (passthrough or VMF mode)
`response strip`	`src/proxy.ts`	strips `x-platform-att` from upstream response before returning

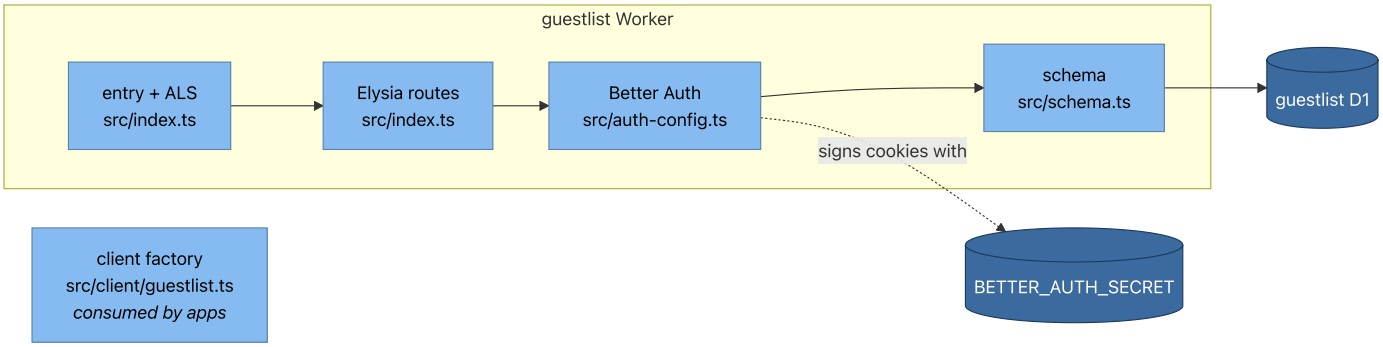
Request lifecycle.



Notes.

- Step 4 is bouncer's only guestlist hop. Whether app pays a second hop is the app's choice (step 9 vs. step 11): identity-only consumers call `platform.getEnvelope()` and pay nothing; routes that need full BA session metadata (`twoFactorEnabled`, `createdAt`, etc.) call `platform.getSession()` which RPCs guestlist with the inbound cookies (~1ms same-colo).
- Browser cookies flow through bouncer to the app unchanged — bouncer's strip-and-stamp only touches `x-platform-*` headers (see §4.1.3). That's what lets `platform.getSession()` authenticate at guestlist without any envelope-to-session synthesis: the cookie is the source of truth, the envelope is the signed origin assertion.
- Step 6's strip is the hygiene rule: never let a client-supplied `x-platform-*` header survive into upstream.
- If step 4's `guestlist.getSession()` returns `null` (no session, or expired), the envelope carries `actor: null` and `session: null`. The envelope is still stamped — every bouncer-forwarded request gets one, so the prod enforcement check is uniform. The verifier enforces the cross-field invariant `actor === null ⟺ session === null`.
- For `auth.require: "user"` routes (per `ROUTES` config), if `actor === null` bouncer short-circuits with a 302 to `identity.<baseDomain>/sign-in?returnTo=<original-url>` before reaching dispatch. The redirect response itself carries no envelope; it's a bouncer-owned response, not an upstream response.

§3.2 guestlist

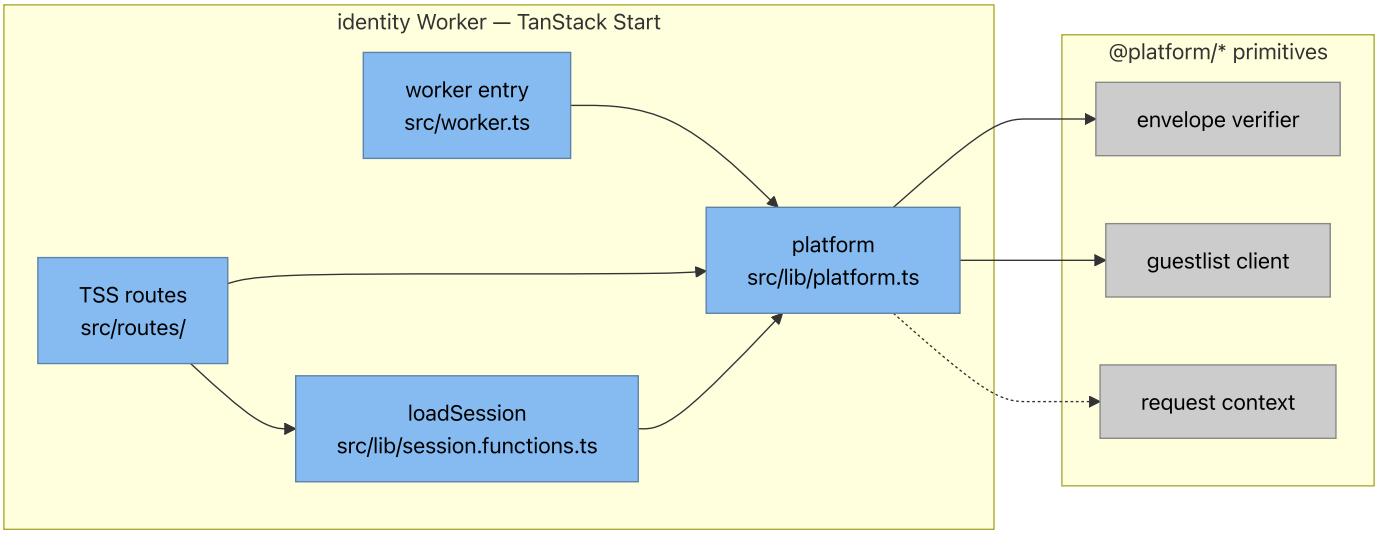


component	role
entry + ALS	wraps fetch in `executionContext` + `withRequestContext`; reads `x-platform-*` as log correlation only
Elysia routes	mounts `/api/auth/*`, `/api/avatar/*`, `/admin/*`, `/user/connections`, `/u/avatar/:refId`, `/health`, `/providers`, `/.well-known/*`
Better Auth	BA instance with plugin set (passkey, twoFactor, oauthProvider, admin, organization once enabled)
schema	Drizzle schema: user, session, account, twoFactor, organization tables
client factory	`createGuestlistClient(...)` — apps consume this to call guestlist over their service binding

Invariants.

1. **Guestlist is the sole holder of `BETTER_AUTH_SECRET`.** No other Worker. Cookie validation lives where the secret lives.
2. **Guestlist is reached only via service binding in the target topology.** It has no public Custom Domain. The bouncer proxies `/api/auth/*` and `/u/*` from `identity.<baseDomain>` to guestlist; that proxy is the only public path to guestlist.
3. **`x-platform-actor-*` headers at guestlist's boundary are log-correlation only.** They never feed authz. Authoritative actor identity at guestlist always derives from the cookie. (Comment that pins this lives at `services/guestlist/src/index.ts` boundary; see §4.1.4.)

§3.3 identity (representative app)



component	role
`worker entry`	`createPlatformStartWorkerEntry({ name: 'identity' })` — one-liner, no per-app TSS plumbing
`platform`	`createPlatformStartApp({ name: 'identity' })` — single wiring file; exposes `getEnvelope`, `getSession`, `getGuestlist`, `sessionMiddleware`, `apiProxyHandlers`, `makeAuthProvider`
`TSS routes`	sign-in, sign-up, account, admin/sessions, `/api/$` catch-all
`loadSession`	`createServerFn` wrapper around `platform.getSession(headers)` — 5 lines, required at app top-level by the TSS compiler

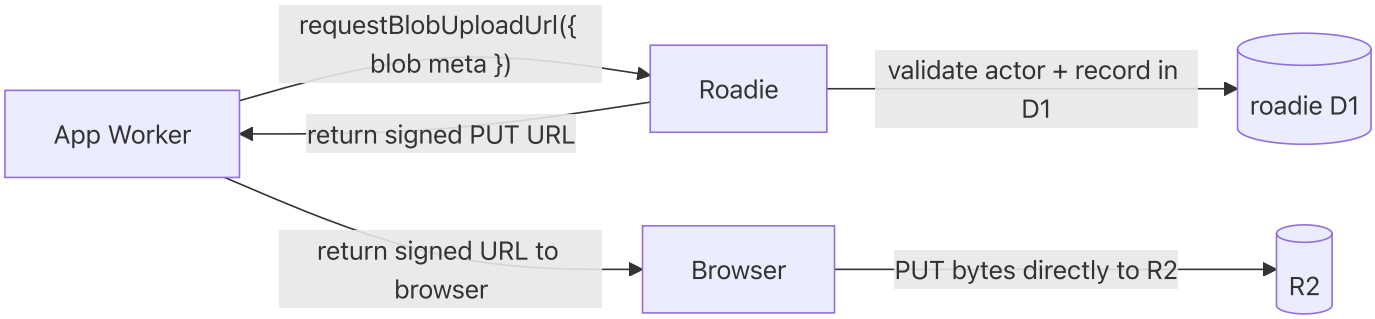
Per-app file budget after cleanup.

```
apps/identity/src/
├── worker.ts                      # 2 lines  — re-exports createPlatformStartWorkerEntry
├── lib/
│   ├── platform.ts                # 3 lines  — createPlatformStartApp({ name })
│   ├── session.functions.ts       # 5 lines  — createServerFn wrapper (required by TSS
compiler)
│   └── (app-specific helpers)
├── routes/
│   ├── api/$.ts                   # 5 lines  — { handlers: platform.apiProxyHandlers }
│   ├── __root.tsx                 # standard TSS root; uses platform.AuthProvider +
loadSession
│   └── ...
├── app-brand.ts                   # APP_PRODUCT_NAME (per-app)
└── wrangler.template.jsonc        # bindings: GUESTLIST, ROADIE?, PROMOTER?
```

~15 lines of platform-glue boilerplate per app. The lower bound TSS allows.

§3.4 roadie

Roadie is a small Worker that owns the platform's blob plane. Apps don't directly hold S3 credentials; they request signed PUT/GET URLs from roadie. Roadie validates the requesting actor (via bouncer's envelope, same as any app), records the blob in its D1 index, and returns presigned R2 URLs.



The signed-meta token pattern: roadie issues a short-lived signed token along with the URL; the URL itself is the R2 presigned URL. The blob index in D1 binds blob id → owner actor → metadata.

§3.5 promoter

Promoter is similar in shape but for email. Apps call `promoter.sendVerificationEmail({ to, code })` over a service binding; promoter formats via templates and dispatches via Resend. Apps never hold `RESEND_API_KEY`.

The signed-meta token in promoter is for outbound-email idempotency — when an app retries a send, promoter dedupes on the token to avoid duplicate emails.

---

§4 — Shared Patterns

The substance. This section documents the cross-cutting patterns every app and service participates in.

§4.1 Security

§4.1.1 The internal header contract: `x-platform-*`

The platform speaks one well-typed internal header family. **Only bouncer translates `cf-*` to `x-platform-*`.** Apps and services never read `cf-*` directly.

```
// @platform/auth/platform-headers (target — moved out of guestlist client)
export const PLATFORM_HEADERS = {
  rid: "x-platform-rid", // canonical request id
  att: "x-platform-att", // bouncer attestation envelope (JWS)
  caller: "x-platform-caller", // calling worker name: "bouncer", "identity", ...
  actor: {
    kind: "x-platform-actor-kind", // "user" | "service"  — log correlation only
    id: "x-platform-actor-id", // string              — log correlation only
  },
} as const;

export interface PlatformRequestContract {
  rid: string; // required on every internal request
  att?: string; // present on bouncer → app traffic; absent on worker → worker traffic
  caller?: string;
  actorKind?: "user" | "service";
  actorId?: string;
}
```

Why this exists:

- **One family, one place to grep.** Every privileged header has the `x-platform-` prefix. Adding a sixth field is a one-file edit.
- **Type-safe.** The constant + interface get imported anywhere the boundary reads or writes.
- **Framework-agnostic.** The contract has nothing to do with TanStack Start, Hono, or any other web framework. It applies to every CF Worker that participates in the platform.
- **Decouples from Cloudflare.** A future port to a non-CF runtime touches bouncer only; everything else already speaks the internal contract.

§4.1.2 Bouncer attestation envelope

Bouncer stamps an Ed25519-signed JWS envelope on every forwarded request — authed or not, public-page or admin-only.

Envelope shape (signed payload).

```
type EnvelopePayload = {
  v: 1; // version
  iss: "bouncer"; // issuer
  iat: number; // epoch seconds (issue time)
  exp: number; // iat + 30 seconds
  host: string; // public host bouncer routed (lowercased, no port)
  actor: EnvelopeActorUser | null; // null for public/optional traffic
  session: EnvelopeSessionData | null; // null iff actor is null
};

type EnvelopeActorUser = {
  kind: "user";
  id: string;
  role: string | null;
  // Common UX fields — enough to render avatar/menu/header
  // without an app-side guestlist hop. Anything beyond this
  // (org membership, 2FA state, etc.) goes through guestlist.
  name?: string;
  email?: string;
  image?: string | null;
};

// Safe subset of BA's full session — id/userId/expiresAt only. No `token`
// (that's the cookie itself), no `ipAddress`/`userAgent` (stale by verify
// time — apps read current values from `cf-connecting-ip` / `user-agent`).
type EnvelopeSessionData = {
  id: string;
  userId: string;
  expiresAt: number; // epoch seconds
};
```

The verifier enforces a cross-field invariant: `actor === null ⟺ session === null`. A signed envelope with one populated and the other null is rejected as `invalid: "actor_session_mismatch"`.

Header (JOSE-compact).

```
x-platform-att: <b64url(joseHeader)>.<b64url(payload)>.<b64url(sig)>

joseHeader = { "alg": "EdDSA", "kid": "gw-2026-05" }
```

Crypto choice. Ed25519 (`alg: "EdDSA"`):

- Bouncer holds the private key as a single wrangler secret (`BNC_ATT_PRIV`).
- Apps hold only the public key set, committed as code in `packages/config/src/bouncer-attestation.ts`. **Public keys are not secrets.** Zero new secrets to manage on apps.
- Verification cost ~0.1 ms per request inside a CF Worker.
- `kid` lets the verifier accept a key set (old + new) during rotation; no flag day.

Anti-replay properties.

- `host` field binds the envelope to a specific destination. A stolen envelope for `thoughts.<baseDomain>` is rejected by `transfers.<baseDomain>`.
- `exp = iat + 30s`. Replay window is 30 seconds, matching the standard JWT compromise — no server-side nonce store required.
- `alg` is hardcoded `"EdDSA"` in the verifier; `alg: "none"` and algorithm-confusion attacks are rejected.
- Unknown `kid` is rejected (no fall-open behavior).

§4.1.3 Header strip + stamp at bouncer

Bouncer is the only worker that writes `x-platform-*` headers onto requests. The strip-then-stamp rule guarantees no client-supplied privileged header survives the proxy boundary, regardless of how the client crafted the request.

```
// services/bouncer/src/proxy.ts (target shape)
function stampUpstreamHeaders(
  request: Request,
  envelope: string,
  actor: { kind: "user"; id: string } | null,
): Request {
  const headers = new Headers(request.headers);

  // STRIP — privileged platform-family headers; client values never survive.
  headers.delete("x-platform-rid");
  headers.delete("x-platform-att");
  headers.delete("x-platform-caller");
  headers.delete("x-platform-actor-kind");
  headers.delete("x-platform-actor-id");

  // STAMP — values authored by bouncer from its request context.
  headers.set("x-platform-rid", getPlatformRid());
  headers.set("x-platform-caller", "bouncer");
  headers.set("x-platform-att", envelope);
  if (actor) {
    headers.set("x-platform-actor-kind", actor.kind);
    headers.set("x-platform-actor-id", actor.id);
  }

  // cf-* headers are LEFT IN PLACE — they're informational (cf-connecting-ip,
  // cf-ipcountry, etc.) and apps are free to consume them. They are NEVER the
  // source of identity or request id downstream.

  return new Request(request, { headers });
}
```

On the response side, bouncer also strips `x-platform-att` from upstream responses before returning to the client. The envelope is internal-only; it should never leak to a browser.

§4.1.4 Verification on the app side

Apps verify envelopes through a single shared primitive in `@platform/auth`.

```
// @platform/auth/createBouncerEnvelopeVerifier (framework-agnostic)
export function createBouncerEnvelopeVerifier(opts: {
  keys: Record<string, string>; // kid → base64-PEM Ed25519 pubkey
  env: "development" | "staging" | "production";
  expectedHost: (req: Request) => string; // typically req.url.hostname
}): (req: Request) => Promise<EnvelopeResult>;

type EnvelopeResult =
  | {
      kind: "valid";
      actor: EnvelopeActorUser | null;
      session: EnvelopeSessionData | null;
    }
  | { kind: "missing" } // no x-platform-att header
  | { kind: "invalid"; reason: string };
```

Dev/prod enforcement matrix. Baked into the verifier — apps don't repeat the logic.

ENVIRONMENT	envelope missing	envelope invalid
`development`	`kind: "missing"` → apps fall back to direct guestlist call	log + same fallback
`staging`	same as dev (warning log)	same as dev (warning log)
`production`	**throw `403 — bouncer_required`**	**throw `403 — envelope_invalid`**

The verifier factory itself **asserts at construction time that `keys` is non-empty when `env === "production"`**. A worker deployed to production with no key set fails to boot, not silently fails open at request time.

§4.1.5 Defense in depth

Two layers always hold:

1. **Bouncer's envelope check** enforces UX policy at the ingress boundary (redirect unauthenticated traffic, attach actor for downstream).
2. **The app's verifier runs as a precondition** on every `platform.getEnvelope()` and `platform.getSession()` call inside loaders/middleware. In prod the verifier throws `403 — bouncer_required` / `envelope_invalid` on missing/tampered envelopes — that's the per-route enforcement; in dev/staging it returns `kind: "missing"` and identity-only consumers see `null` (apps that need full session still cookie-authenticate against guestlist the same way).

A bouncer misconfiguration that forgot to mark a route as `auth.require: "user"` is caught by the app's own session check. A missing envelope in prod is caught by the verifier (403). Both must hold; neither replaces the other.

§4.2 User identity & session access

Apps never deal with JWS, headers, or envelopes directly. The platform exposes **two distinct functions** that return **two distinct types** — the envelope is intentionally not synthesized into a "session" by the kit, because the envelope's narrow payload can't honestly satisfy Better Auth's full plugin-merged session shape. Picking which one to call is the only thing app code thinks about.

```
// Fast path — narrow, signed, no I/O.
// Use for auth gates, log correlation, header chrome.
const env = await platform.getEnvelope(headers);
if (env) {
  env.actor.id;
  env.actor.role;
  env.actor.email; // optional
  env.session.expiresAt; // epoch seconds
}

// Full path — BA-inferred `PlatformSession`, ~1ms service-binding hop.
// Use for plugin-extended fields (twoFactorEnabled, createdAt, username, etc.).
const session = await platform.getSession(headers);
if (session) {
  session.user.id;
  session.user.twoFactorEnabled; // BA admin/2FA plugin field
  session.session.token; // BA-managed cookie value
}
```

Both calls run the envelope verifier as a precondition, so production's "every request must originate through bouncer" guarantee holds either way. The split is purely about whether the caller is willing to make a guestlist service-binding RPC.

Type relationship. `PlatformSession` is a strict superset of `EnvelopeData`'s relevant fields — same `user.id`, same `user.email`, etc. Anywhere envelope data is enough, the full session also works; anywhere the full session is required, the envelope can't substitute (it doesn't carry the BA-plugin extras). Apps don't have to choose one model — they pick per call site:

- Route loader for a public-page header → `getEnvelope()`.
- Route loader for `/account` (reads `twoFactorEnabled`, `emailVerified`) → `getSession()`.
- Server-fn auth gate (just needs `user.id`) → `getEnvelope()`.
- Admin impersonation panel (reads `session.impersonatedBy`) → `getSession()`.

Cookies still flow through. Bouncer's strip-and-stamp doesn't touch the `Cookie` header. Apps' guestlist client (`getCookies()` from `@tanstack/react-start/server` → service-binding RPC) authenticates against the same session the browser sent. The envelope is the signed bouncer-origin assertion; the cookie remains the actual auth credential.

The legacy `createSessionReader` fast-path that distributed `BETTER_AUTH_SECRET` to every app is **removed**. The envelope replaces it; nothing about app code changes except no longer having a `BETTER_AUTH_SECRET` to rotate per app.

§4.3 Cross-worker communication

All worker-to-worker communication is via Cloudflare service bindings. **Never via public HTTP.** Three patterns:

§4.3.1 App → guestlist

Apps wire `createGuestlistClient(...)` once via `createPlatformStartApp`. The returned client looks like:

```
platform.getGuestlist().getSession();
platform.getGuestlist().listUserSessions({ userId });
platform.getGuestlist().admin.banUser({ userId, reason });
```

The factory sets the `x-platform-caller` header to the app's name, forwards `x-platform-rid` from the active request context, and passes through cookies on calls that need them (e.g., `getSession`). Guestlist reads these headers as log-correlation context only.

§4.3.2 App → roadie / promoter

Same pattern. Each service exposes a typed RPC client (`createRoadieClient` , `createPromoterClient`) plumbed through the same `createPlatformStartApp` -returned object:

```
platform.getRoadie().requestUploadUrl({ blobMeta });
platform.getPromoter().sendVerificationEmail({ to, code });
```

§4.3.3 Bouncer → app

Bouncer uses the raw `Fetcher` from its service binding (no typed client) because bouncer is proxying arbitrary HTTP, not making typed RPCs. The fetcher dispatches the (modified) `Request` to the upstream worker via `upstream.fetch(request)`. Same call pattern handles WebSocket upgrades (see §4.6).

§4.4 Request lifecycle & correlation

Every Worker opens an ALS request scope at its `fetch` boundary. The scope carries `{ requestId, actorKind, actorId, callerApp }` and is read by canonical-log emission, service-client headers, and middleware.

```
// Every worker entry follows this shape:
export default {
  async fetch(request, env, ctx) {
    return withRequestContext({ requestId: extractPlatformRequestId(request) }, () =>
      withRequestLog({ service: "<name>" }, request, async (log) => {
        // ... actual handler
      }),
    );
  },
} satisfies ExportedHandler<Env>;
```

For TSS apps, `createPlatformStartWorkerEntry({ name })` does this once and forwards into TSS's server entry. For non-TSS apps, the pattern is the same three lines, hand-written.

`extractPlatformRequestId(request)` fallback chain. A tiered lookup that works in every topology:

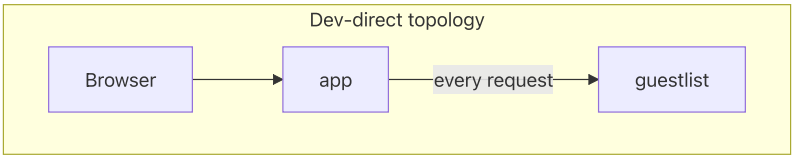
```
export function extractPlatformRequestId(req: Request): string {
  // 1. Internal contract — set by bouncer, or by an upstream platform caller.
  const internal = req.headers.get(PLATFORM_HEADERS.rid);
  if (internal) return internal;
  // 2. Only bouncer gets here when CF is in front — it reads cf-request-id
  //    via its dedicated extractor. Other workers don't, because they're
  //    only reached via service binding from another platform worker that
  //    has already set x-platform-rid.
  // 3. Dev-app-alone, or any path without an upstream platform caller — mint.
  return crypto.randomUUID();
}
```

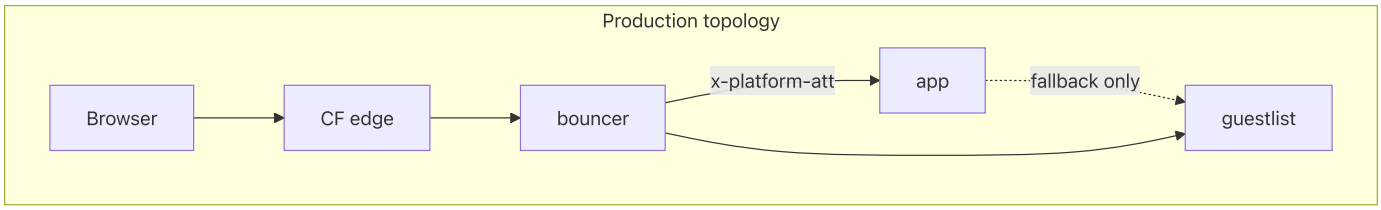
Bouncer itself has a slightly different entry — it reads `cf-request-id` because it sits behind CF's edge and the public Internet's "client" doesn't speak the platform contract.

§4.5 Dev/prod parity & the dev-direct topology

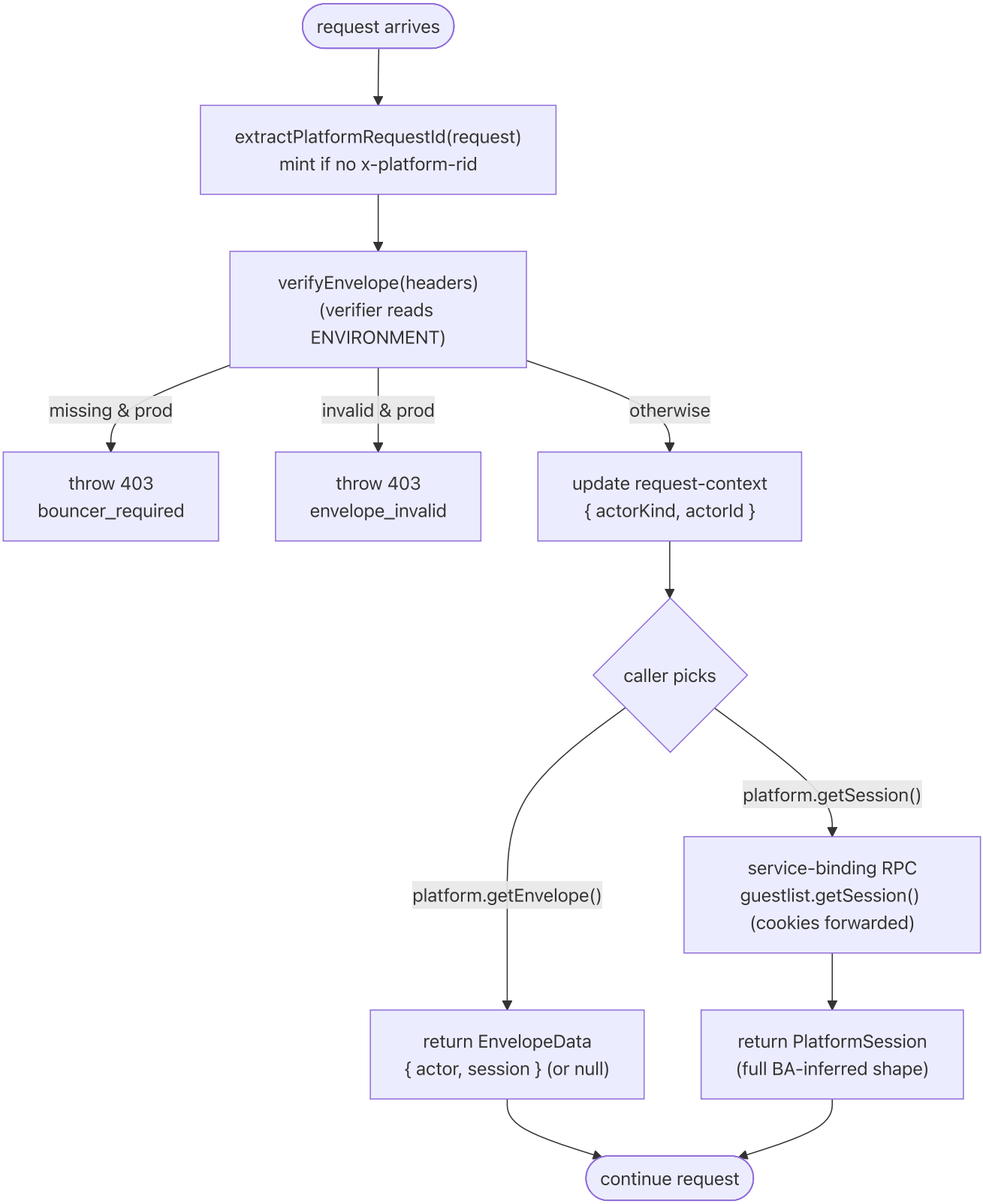
Apps must run alone in development without bouncer in front. The same code runs in both topologies; the behavioral difference is encoded in `ENVIRONMENT`.

The two topologies.





What the app does differently.



The verifier is the entire dev/prod story. Apps don't have `if (ENVIRONMENT === ...)` branches in their own code — `getEnvelope()` and `getSession()` both feed off the same precondition. Which one the caller invokes is a per-route choice based on whether BA-extended fields are needed.

Why this is safe in production.

1. `ENVIRONMENT === "production"` is a strict string compare, no truthy fallback.
2. `BOUNCER_ATTESTATION_KEYS` is asserted non-empty when constructing the verifier in production — a misconfigured worker fails to boot, not silently fails open.
3. Apps have no public Custom Domain in production; they're reached only via service binding from bouncer. CI enforces this via wrangler-template linting (see §6.3).

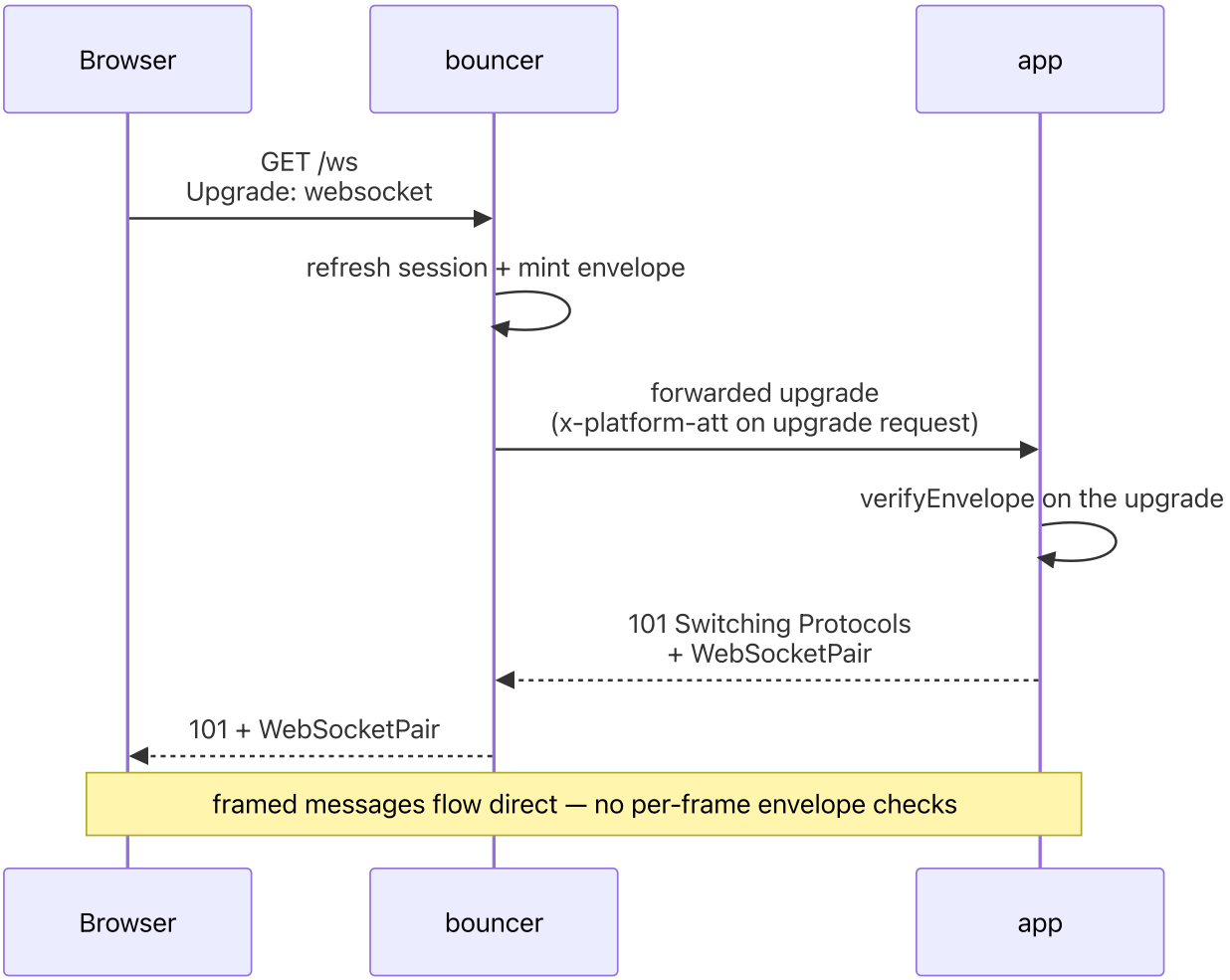
Why this works in development.

1. No CF edge → no `cf-request-id` → app's entry shim mints one.
2. No bouncer → no `x-platform-att` → verifier returns `kind: "missing"` → `getEnvelope()` returns `null`; `getSession()` proceeds to its guestlist service-binding hop using cookies on the inbound request.
3. App's `GUESTLIST` service binding is wired in every environment's `wrangler.template.jsonc` (including dev) — `getSession()` always has somewhere to land.

A developer pulls the repo, runs `bun install`, `bun run bootstrap`, `cd apps/identity && bun run dev`, and sign-in works against a local guestlist — no bouncer involved.

§4.6 WebSocket upgrades

Bouncer proxies WebSocket upgrades transparently. The attestation lives on the upgrade request headers; framed messages are not annotated.



The actor is "frozen" at upgrade time. If the user's session is revoked mid-connection, the WebSocket stays open until the app closes it on its own logic. This matches the existing semantics of `getSession()` in route loaders — point-in-time identity.

§4.7 Canonical logging

Every Worker emits **one canonical log line per request** at the boundary. Per-request `log.add({...})` calls accrue fields into a single ALS-scoped builder; the final line is JSON, contains the resolved actor + caller + request id + outcome + timings.

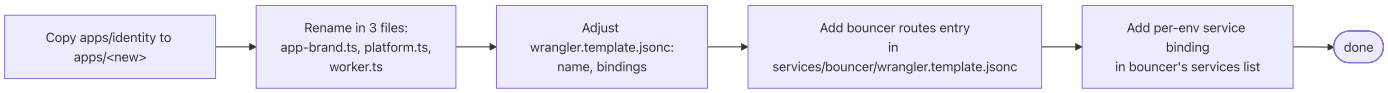
```
withRequestLog({ service: "identity" }, request, async (log) => {
  log.add({ route: "/dashboard" });
  // ... handler
  log.outcome("ok"); // or "internal_error", "http_404", etc.
  // single JSON line emitted on scope close
});
```

The shape is consistent across bouncer, guestlist, apps, roadie, promoter. Log aggregation (Workers Logs, Logpush) keys off `request_id` to join lines from different services for the same end-user request.

§5 — Adding a new app

The 90% case is a TanStack Start app. The non-Start path is a documented 10% case for when you need to drop in a Hono or plain-Workers app.

§5.1 Adding a TanStack Start app



The required edits.

1. `apps/<new>/src/lib/platform.ts` — single change is the `name` value:

```
import { createPlatformStartApp } from "@platform/kit/react-start";
export const platform = createPlatformStartApp({ name: "<new>" });
```

2. `apps/<new>/src/lib/session.functions.ts` — copy verbatim (TSS-compiler constraint):

```
import { createServerFn } from "@tanstack/react-start";
import { getRequestHeaders } from "@tanstack/react-start/server";
import { platform } from "@/lib/platform";
export const loadSession = createServerFn({ method: "GET" }).handler(() =>
  platform.getSession(getRequestHeaders()),
);
```

3. `apps/<new>/src/worker.ts` — single change is the `name` value:

```
import { createPlatformStartWorkerEntry } from "@platform/kit/react-start";
export default createPlatformStartWorkerEntry({ name: "<new>" });
```

4. `apps/<new>/src/routes/api/$.ts`:

```
import { createFileRoute } from "@tanstack/react-router";
import { platform } from "@/lib/platform";
export const Route = createFileRoute("/api/$")({
  server: { handlers: platform.apiProxyHandlers },
});
```

5. `apps/<new>/src/app-brand.ts` — set `APP_PRODUCT_NAME`.
6. `apps/<new>/wrangler.template.jsonc` — name, service bindings to `GUESTLIST` (+ `ROADIE`, `PROMOTER` if used).

7. `services/bouncer/wrangler.template.jsonc` — add a `services` binding entry for the new app and a `ROUTES` entry mapping `<new>.{{BASE_DOMAIN}}` to the new binding.

Total: ~25 lines across 7 files, of which ~3 are app-name string substitutions.

§5.2 Adding a non-TanStack-Start app

A non-Start app composes the same framework-agnostic primitives directly. The canonical entry looks like:

```
import { createBouncerEnvelopeVerifier, PLATFORM_HEADERS } from "@platform/auth";
import { createGuestlistClient } from "@platform/guestlist-service/client";
import { withRequestContext, extractPlatformRequestId } from "@platform/kit/request-context";
import { withRequestLog } from "@platform/kit/log";
import { BOUNCER_ATTESTATION_KEYS } from "@platform/config";

const verifyEnvelope = createBouncerEnvelopeVerifier({
  keys: BOUNCER_ATTESTATION_KEYS,
  env: env.ENVIRONMENT,
  expectedHost: (req) => new URL(req.url).hostname.toLowerCase(),
});

export default {
  async fetch(request: Request, env: Env, ctx: ExecutionContext) {
    const rid = extractPlatformRequestId(request);
    return withRequestContext({ requestId: rid }, () =>
      withRequestLog({ service: "<name>" }, request, async (log) => {
        const result = await verifyEnvelope(request);
        // result.kind === "valid"   → result.actor and result.session are
        //                             both present, or both null (verifier
        //                             enforces actor ⟺ session invariant)
        // result.kind === "missing" → no x-platform-att (dev/staging only;
        //                             prod throws 403 inside the verifier)
        // result.kind === "invalid" → tampered or expired (same)
        // For full BA session metadata, call guestlist over the service
        // binding the same way TSS apps do — cookies on the inbound request
        // flow through bouncer unchanged.

        // ... your framework's routing here (Hono, plain handlers, etc.)
        return new Response("ok");
      }),
    );
  },
} satisfies ExportedHandler<Env>;
```

Everything else (config, secrets, bouncer wiring) is the same as the TSS path. The app gets a wrangler template, a service binding to guestlist, a bouncer route — identical infrastructure.

---

§6 — Config & Secrets

§6.1 Config split

A small set of files owns the platform's branding and deploy surface.

File	What lives here	When you edit it
`packages/config/src/brand.ts`	`brand.{name, short, supportEmail}`; `cookies.prefix`; `auth.{providerId, passkeyRpName, twoFactorIssuer}`	Once per fork. Sets the visible identity of the platform.
`packages/config/src/deploy.ts`	`baseDomain`, `devDomain`, `workersDevSubdomain`, `cloudflareAccountId`, `d1.{guestlist,roadie}.{local,staging,production}`	Once per fork (and once per env when D1 ids change).
`packages/config/src/bouncer-attestation.ts`	`BOUNCER_ATTESTATION_KEYS` — `kid → public-key` map	On Ed25519 key rotation (see §6.4).
`apps/<app>/src/app-brand.ts`	`APP_PRODUCT_NAME` per app (each app is its own product)	Once per app.

These files cover every brandable / deployable constant in the platform. **Anything that needs branding reads from `@platform/config`** (or from the app's local `app-brand.ts`). No platform-name literal lives outside these files.

§6.2 Wrangler template rendering

Per-service `wrangler.template.jsonc` files are substituted at render time into `wrangler.jsonc` (gitignored). Substitution happens in `scripts/render-wrangler.ts` against tokens like `{{BASE_DOMAIN}}`, `{{DEV_DOMAIN}}`, `{{D1_GUESTLIST_LOCAL}}`, etc., sourced from `packages/config/src/deploy.ts`.

`bun run dev`, `bun run bootstrap`, and `bun run build` all chain `bun scripts/render-wrangler.ts` first, so re-rendering is automatic on the common paths. After editing `deploy.ts`, regenerate per-service worker types: `cd <service> && bun run types`.

§6.3 Secrets matrix

Secret	Per env	Holder	Purpose
`BETTER_AUTH_SECRET`	yes	**guestlist only**	Better Auth cookie signing. Single holder by design — apps no longer hold it.
`BNC_ATT_PRIV`	yes	**bouncer only**	Ed25519 private key for envelope signing. Single secret, single rotation point.
`RESEND_API_KEY`	yes	promoter	Outbound email.
`S3_ACCESS_KEY_ID` / `S3_SECRET_ACCESS_KEY`	yes	roadie	R2 SigV4 credentials.
OAuth client id/secret pairs	yes	guestlist	Google, Microsoft, GitHub, etc. — wired conditionally.

Things that are **not secrets** (public values, committed to repo or `vars`):

- `BETTER_AUTH_URL`, `IDENTITY_URL`, `AUTH_DOMAIN` — public URLs.
- `EMAIL_FROM` — the From: address (the API key is the secret).
- `BOUNCER_ATTESTATION_KEYS` — public-key set; lives in `packages/config/src/bouncer-attestation.ts`.
- D1 ids, account ids — non-sensitive identifiers.

§6.4 Rotation runbooks

`BETTER_AUTH_SECRET` (guestlist)

Single-holder rotation is one command and one moment of session invalidation:

```
echo "$(openssl rand -base64 32)" | \
  bunx wrangler secret put BETTER_AUTH_SECRET --env production --cwd services/guestlist
```

Every active session is invalidated. Users sign in again. No coordination across services needed (because the secret only lives in guestlist).

`BNC_ATT_PRIV` (bouncer, with overlap window)

Public-key publication is via committed code, so rotation is a code change + a secret rotation in sequence:

1. Generate keypair: `openssl genpkey -algorithm ed25519 -out priv.pem; openssl pkey -in priv.pem -pubout -out pub.pem`.

2. **PR #1** — add new `kid: pub.pem` entry to `BOUNCER_ATTESTATION_KEYS` (both old + new in the set). Deploy all apps.
3. **PR #2** — `wrangler secret put BNC_ATT_PRIV` on bouncer with the new private key. Update `BNC_ATT_KID` env var on bouncer to the new kid. Deploy bouncer. From this moment, bouncer signs with the new kid; apps accept both during the overlap.
4. **PR #3** — drop the old `kid` from `BOUNCER_ATTESTATION_KEYS`. Deploy apps. Old envelopes (max 30s lifetime) are gone by then.

No flag day. The overlap window is bounded by the envelope's `exp` (30s), not by deploy coordination.

Appendix A — Topology decision matrix

When the platform is running, one of these topologies is in effect.

Topology	Where	What's in front of apps	Envelope present	Identity resolution
Production	`<host>.<baseDomain>` via CF Custom Domain	bouncer	always	`getEnvelope()` = signed payload, no I/O. `getSession()` = guestlist service-binding RPC (cookies passed through bouncer). Verifier enforces.
Staging	`<host>-staging.<workersDevSubdomain>.workers.dev`	bouncer	always	same as prod (or dev fallback, see §4.1.4)
Dev (full)	`<host>.<devDomain>` via portless	bouncer (run locally)	always	same as prod
Dev-direct (app-alone)	`<host>.<devDomain>` via portless, or `127.0.0.1:<port>` via wrangler dev	nothing	never	`getEnvelope()` returns `null` (verifier kind `missing` in dev). `getSession()` still hits guestlist over service binding via inbound cookies.

The `getEnvelope()` fast path costs nothing — JWS verify is sub-millisecond. The `getSession()` path costs one service-binding RPC, same in every topology that has a guestlist binding.

Appendix B — Glossary

- **Guestlist** — the Worker that owns the user database and Better Auth. The sole authority on session validity.
- **Envelope** — the bouncer attestation, a JWS-compact value carried in `x-platform-att`. Signed with Ed25519. Lives 30 seconds. Payload: `{ actor, session, host, iat, exp, ... }` — narrow safe projection of the resolved BA session.
- **`EnvelopeData`** — the verified envelope's `{ actor, session }` pair, returned by `platform.getEnvelope()`. Narrow, signed, no I/O.
- **`PlatformSession`** — Better Auth's full plugin-inferred session, returned by `platform.getSession()` after a guestlist service-binding hop. Strict superset of `EnvelopeData`'s relevant fields.
- **Bouncer** — the single public-ingress Worker. Translates `cf-*` → `x-platform-*`. Mints envelopes.
- **Identity** (the app) — the reference TanStack Start app. Owns sign-in, sign-up, account, admin sessions surface.
- **Kid** — key id. A short string in the JWS header identifying which Ed25519 key signed the envelope. Allows key-set rotation.
- **Platform contract** — the `x-platform-*` header family. The only privileged header set apps and services read.
- **Promoter** — the Worker that owns outbound email. Wraps Resend.
- **Roadie** — the Worker that owns blob storage. Wraps R2 with signed-URL minting.
- **VMF** — virtual microfrontend; bouncer's `mode: "vmf"` rewrites HTML/CSS/Location/cookies when mounting an upstream app under a path prefix. Default mode is `"passthrough"` (no rewriting), used for full-subdomain apps.